Tech Club Event Request System — Product Specification

The League of Amazing Programmers Draft v0.4 — March 2026 DRAFT — FOR INTERNAL REVIEW

---

1. Overview

1.1 Problem

The League currently schedules free Tech Club events manually — contacting libraries, schools, and community centers to arrange locations, then coordinating instructors, equipment, and promotion for each event. This process is time-consuming and doesn't scale. Meanwhile, demand from parents, schools, and youth groups is growing to the point where a self-service request system is viable.

1.2 Solution

A web application that lets parents, teachers, and group leaders request Tech Club events for their communities. The system matches requests against instructor availability and geography, creates a structured coordination workflow, and handles registration and promotion for confirmed events.

1.3 Goals

- Shift event initiation from League outreach to community demand
- Reduce manual coordination work for admins
- Give instructors/volunteers control over their own availability and preferences
- Provide a clear pipeline from request to confirmed event to registration
- Integrate with existing systems (Pike13, Meetup, main website, Asana, email)

1.4 Non-Goals (v1)

- Payment processing (donations are handled via Give Lively link)
- Replacing Pike13 for paid class enrollment
- Multi-session courses (these involve instructor assignment for a full series and are paid programs — a different workflow)
- Managing the ongoing curriculum or class content (that stays on jointheleague.org)

---

2. Features

The Tech Club Event Request System is the front door for anyone who wants to bring a free League tech class to their community. A parent, teacher, or scout leader visits the League's website, finds a class they want — Python, robotics, game design — and clicks a button to request it for their group. The system checks whether an instructor is available in their area, shows them open dates, and walks them through a short intake form. Once they submit the request, the system creates a shared email thread connecting the requester with League staff (Jed, Eric, and others) and the matched instructor so they can work out the details: venue, logistics, promotion.

When the event is ready to go, the requester gets a shareable link they can send to families in their group. Those families vote on which dates work for them and register their kids. If enough people commit to a date, the event is confirmed and everyone gets a calendar invite. For public events, the system creates a Meetup listing instead and lets Meetup handle the RSVPs. Behind the scenes, League staff track the whole pipeline in Asana, and an AI agent quietly reads the email threads to keep the task board current.

The system doesn't hold its own content — class descriptions, age ranges, and topics all come from jointheleague.org. It doesn't handle payments either; donations go through Give Lively. What it does is take the manual work of finding venues, matching instructors, coordinating schedules, and tracking interest, and turn it into a structured workflow that scales.

2.1 Feature List

Class Catalog & Request Intake

- Browse requestable Tech Club classes sourced from jointheleague.org
- Request a specific class for your group directly from the class page
- Geographic availability check — enter a zip code, see whether instructors can reach you
- Instructor availability calendar — view open dates for your area and topic, select preferred dates
- Intake form collecting group details, site information, and marketing capability
- Optional external registration URL for hosts that have their own registration system
- Donation encouragement with estimated event cost and link to Give Lively

Instructor/Volunteer Management

- Pike13 OAuth login for instructors and admins
- Instructor profile: topics they teach, home zip, travel range or specific service zip codes
- Availability signaled through Pike13 appointments (instructors use Pike13's existing calendar UI)
- Automatic matching of instructors to requests by topic, geography, and availability
- Instructor consent: matched instructors must accept a request before they're assigned; the system sends reminders if they haven't responded

Equipment Reservation

- Integration with the League's inventory system to check and reserve equipment (robots, micro:bits, laptops, etc.) needed for a class
- Equipment availability is a scheduling constraint — if the gear isn't free on a given date, that date isn't offered

Event Coordination

- Per-request email address for threaded communication between requester, admins, instructor, and other contacts (e.g. school principal)
- AI-powered email extraction: action items, status updates, and decisions pulled from email threads and synced to Asana (read-only — AI does not send emails)
- Asana integration: each request becomes a task with status tracking, custom fields, and pipeline management
- Request status lifecycle: new → discussing → dates proposed → confirmed → completed/cancelled

Registration & Interest Gathering

- Private events: shareable registration link with date voting ("check all dates you can attend") and minimum headcount threshold
- Public events: automatic Meetup event creation with RSVPs handled by Meetup
- Public events with external registration: Meetup description includes a prominent "Host Registration" hyperlink near the top directing attendees to the host's own registration system
- Periodic registration digest emails sent to the event email thread with names, kid counts, and totals — includes a soft request for the host to share their own registration total
- Date finalization (private events): when a date clears the minimum headcount, it's selected; if multiple clear, highest count wins or admin picks
- iCal invite delivery to confirmed attendees (private events)
- Notifications to registrants who selected a different date than the one confirmed (private events)

Event Types

- Private: closed group (scout troop, school, youth group). Registration via in-app link. Off Meetup.
- Public: open to anyone. Registration via Meetup. Promoted through League's Meetup group. Host may optionally have their own registration system linked from the Meetup page.
- Both types share the same request and coordination flow; they diverge at registration.

Admin Tools

- Dashboard to view and manage all requests
- Configure per-event parameters (minimum headcount, voting deadline)
- Add or edit external registration URL for public events
- Confirm or cancel events
- Asana pipeline management with bidirectional sync (status changes in either system propagate)

2.2 User Roles

Role	Who	Capabilities
Requester	Parent, teacher, scout leader, group organizer	Browse requestable classes. Submit event requests. Provide optional external registration URL. Participate in email coordination. Share registration/interest links. View event status.
Instructor	Volunteer or staff member	Log in via Pike13. Set topic preferences and geographic range in app. Signal availability via Pike13 appointments. Get matched to requests. Participate in email coordination.
Admin	Jed, other League staff	All instructor capabilities. View and manage all requests. Configure event parameters (min headcount, etc.). Add/edit external registration URL. Confirm/cancel events. Manage Asana pipeline.
Attendee	Kids/families registering for a confirmed event	Click registration link. Vote on dates (if multiple, private events only). Register interest/attendance. Receive iCal invite (private events).

---

3. User Flows

3.1 Requester Flow: Submit a Request

Entry point: Parent/teacher is on jointheleague.org. They navigate to the Tech Club page or a page listing requestable classes. Each requestable class has a "Request this free class for your group" button.

1. Requester clicks the request button on a class page. This links to the event request app with the class slug as a parameter.
2. Event app landing page shows class details (pulled from content.json): description, age range, topics covered, typical duration, what equipment is needed.

3. Requester enters zip code. System checks instructor coverage for that area and class topic. If no instructors are available in range, the system says so and suggests they check back or try a different class.
4. If instructors are available, system shows a calendar view of available dates (aggregated from matched instructors, no instructor names shown). Requester selects one or more preferred dates.
5. Requester completes the intake form: name, email, group type, expected headcount, site control, site readiness, marketing capability, optional external registration URL (for hosts with their own registration system), any additional contacts (e.g. school principal email).
6. System displays donation information: estimated cost to run the event, strong encouragement to donate, link to Give Lively.
7. Requester confirms the request. System saves the request in unverified status and sends a verification email with a tokenized link to the requester's email address.
8. Requester sees a holding page: "Check your email to verify your request. If you don't see it, be sure to check your spam folder." The request has a one-hour verification window.
9. **If verified within one hour:** the request moves to new status. System generates a dedicated email address, creates an Asana task, and sends notification emails to admins and matched instructor(s).
10. **If not verified within one hour:** the unverified request is purged from the database. If the requester is still on the holding page, it updates to show that the request has expired with a link to start a new request. If the requester clicks the expired verification link later, they see the same expiration message with a link to start over.

3.2 Coordination Flow

1. All parties communicate via the dedicated email address. Requester, admins (Jed + others), matched instructor, and any additional contacts (principal, etc.) are all on the thread.
2. AI agent monitors the email thread and extracts structured information: confirmed dates, location details, action items, status changes. Updates Asana task and event request record accordingly.
3. Admin or the system moves the request through statuses: discussing → dates_proposed → confirmed.
4. Once confirmed: date, instructor, and location are locked in. The event is ready for registration.

3.3 Registration Flow: Private Events

1. System generates a shareable registration link for the confirmed event.
2. Requester distributes the link to their group (scout troop, school families, etc.).
3. Attendees click the link and land on a registration page showing: class description, location, date(s), and a registration form.
4. If multiple candidate dates are still open, attendees select all dates they can attend ("check all that work for you").

5. Attendee provides: name, email, number of kids attending.
6. System tracks registrations per date. When a date meets the minimum headcount threshold, that date is auto-selected (or flagged for admin confirmation). The requester and admin are notified.
7. If multiple dates clear the threshold, the date with the highest registration count is selected (or admin picks).
8. Once a date is finalized: all registrants who selected that date receive an iCal invite via email. Registrants who didn't select that date get a notification that a different date was chosen.
9. If no date meets the minimum headcount by a deadline (configured per event), the requester is notified and the event may be cancelled or rescheduled.
10. Event is added to the League's internal Google Calendar.

3.4 Registration Flow: Public Events

1. Once confirmed, the system creates a Meetup event via the Meetup API, associated with the appropriate Meetup group (e.g. the-league-tech-club).
2. If the event request has an external registration URL, the Meetup event description includes a prominent hyperlink near the top (e.g. "This event requires registration with the host: Host Registration") before the standard class description and details.
3. The Meetup event link is shared with the requester and displayed in the event app.
4. Registration and RSVPs are handled by Meetup. When an external registration link is present, Meetup serves primarily as a discovery and promotion channel; the host's registration system is the primary signup method.
5. The system periodically syncs RSVP data from Meetup for tracking, up until the day of the event.
6. The system sends periodic registration digest emails to the event's dedicated email thread containing registrant names, number of kids, and totals. When an external registration link is present, the digest includes a request for the host to share their own total registration count including League registrations.
7. Event is also added to the League's internal Google Calendar.

3.5 Instructor Flow

1. Instructor logs into the event app via Pike13 OAuth.
2. On first login, they set up their profile: topics they teach, home zip code, maximum travel time or explicit list of service zip codes.
3. Instructor signals availability by creating appointments in Pike13 (using Pike13's existing UI and calendar). The event app reads these via Pike13 API.
4. When a request matches their profile (topic + geography + availability), they receive an email notification asking them to accept or decline the assignment.

5. If the instructor doesn't respond, the system sends reminders. After a configurable timeout, the system moves on to the next matched instructor.
6. Once accepted, the instructor is added to the request's email thread and can view the event in the app.

3.6 Matching Algorithm

When a requester enters a zip code and class topic, the system finds candidate instructors:

1. Filter by topic: instructor's topics[] must include the requested class slug (or a parent topic that covers it).
2. Filter by geography: if instructor has explicit service_zips, check if the requested zip is in the list. Otherwise, calculate distance from instructor's home zip centroid to requested zip centroid and compare against max_travel_minutes (using estimated drive time with some accounting for time of day). *Note: Zip centroid distance is the v1 approach. May upgrade to a routing API later if needed.*
3. Filter by availability: query Pike13 for the instructor's available appointment slots within a relevant date range.
4. Aggregate: available dates across all matching instructors. Present to requester as a calendar (without instructor names).

---

4. External Systems

4.1 Integration Map

System	Role	Integration
Pike13	Identity & availability	OAuth for instructor/admin login. API read for instructor availability appointments. API write to book the instructor when assigned to an event, so they see it on their Pike13 calendar.
jointheleague.org	Content source & entry point	JSON API provides class catalog (content.json). Class pages link to event request app. Requestable classes flagged in content data.
Meetup	Public event promotion	API write to create events for public Tech Club sessions. Event description includes external registration link when provided. RSVP managed by Meetup. Periodic RSVP sync to event app.
Amazon SES	Email routing	Inbound routing for per-request email addresses. Outbound for notifications, registration digests, and iCal invites.

System	Role	Integration
Asana	Task/pipeline tracking	API to create and update tasks. One project for all requests with custom fields for status, instructor, dates, etc.
Give Lively	Donations	Outbound link only. No API integration.
Google Calendar	Internal event coordination	Private/internal events added to a shared League calendar. Attendees receive iCal invites.

4.2 Email System

Each event request gets a unique email address, e.g. `request-4827@events.jointheleague.org`. This address is the hub for all coordination related to that request.

- **Inbound:** Amazon SES receives mail to this address and routes it to the app via SNS notification or Lambda.
- **Outbound:** The app sends via SES from this address so replies thread correctly.
- **Participants:** Requester, admins, matched instructor(s), additional contacts (principal, etc.). The requester can add participants (e.g. CC the principal) during the intake form or later.
- **Registration digests:** For public events, the system periodically sends a registration summary to the thread with registrant names, kid counts, and totals. When an external registration link is present, the digest includes a soft ask for the host to share their total count.
- **Storage:** All messages are stored in the app's database, linked to the event request.

4.3 AI Email Processing

An AI agent reads each inbound message on the thread and extracts:

- Status-relevant information (e.g. "we've confirmed the library for March 22" → update location and date)
- Action items (e.g. "can you send the flyer template?" → create Asana subtask)
- Decisions (e.g. "let's go with the Saturday session" → flag for admin confirmation)
- Sentiment/risk signals (e.g. requester going cold, repeated rescheduling)
- Host registration counts (e.g. host replies with "we have 25 registered total" → update tracking)

The AI does **not** send emails to participants. It only extracts information and updates internal systems (Postgres, Asana). Admin review is required for any status changes the AI flags.

4.4 Asana Integration

One Asana project holds all event requests. Each request is a task.

Field	Type	Notes
Requester Name	Text	
Class Requested	Text	Class title from content.json
Group Type	Dropdown	School, Girl Scouts, BSA, Library, Other, Public
Zip Code	Text	
Expected Headcount	Number	
Status	Dropdown	New, Discussing, Dates Proposed, Confirmed, Completed, Cancelled
Instructor	Text	Assigned instructor name
Confirmed Date	Date	
Location	Text	Venue name and address
Event Type	Dropdown	Private, Public
External Registration	URL	Host's registration link (if provided)
Registration Count	Number	Current registrations (updated by system)
Event App Link	URL	Link back to the request in the event app

Jed manages the pipeline in Asana — moving tasks between statuses, adding notes, assigning follow-ups. The system keeps the task in sync bidirectionally where possible.

4.5 Content Integration

The event app does not maintain its own class content. All class information is sourced from jointheleague.org's content.json.

A new `requestable` flag in content.json marks which classes can be requested:

- Classes with the flag appear on the "Request a Tech Club" listing page on jointheleague.org
- Each class page gets a "Request this free class for your group" button linking to the event app
- The event app reads class metadata (title, description, age range, topics, typical duration, equipment needs) from content.json at request time

When a public event is created, the system uses groups.json to map class subgroups to Meetup groups (e.g. tech-club/robot → the-league-tech-club) for event creation.

5. Data Model

Primary data store is PostgreSQL. Pike13 is the identity provider. The main website's content.json is the source of truth for class definitions.

5.1 Instructor Profile

Stored in Postgres, keyed to Pike13 user ID.

- `pike13_user_id` — from OAuth
- `display_name` — first name or first name + last initial
- `topics[]` — list of class topics they're willing to teach (references class slugs from content.json)
- `home_zip` — their base location
- `max_travel_minutes` — how far they'll drive
- `service_zips[]` — optional explicit list of zip codes they'll serve (overrides radius)
- `active` — whether they're currently taking requests

5.2 Instructor Availability

Read from Pike13 API. Instructors create appointments in Pike13 to signal when they're available. The event app reads these and presents them as bookable windows.

- Pulled from Pike13 appointment data
- Matched against request geography and topic
- Displayed to requesters as available date/time slots

5.3 Event Request

Created when a requester submits the intake form.

- `request_id`
- `class_slug` — which class was requested (from content.json)
- `requester_name`, `requester_email`
- `group_type` — school, scout troop (Girl Scout, BSA), library, other youth group, public
- `zip_code`
- `location_address` — may be provided later during coordination
- `expected_headcount`

- `site_control` — does the requester control the venue?
- `site_readiness` — how much work to get the site ready?
- `marketing_capability` — can the requester help promote?
- `preferred_dates[]` — dates selected from available instructor windows
- `email_address` — the per-request routing address (e.g. request-{id}@events.jointheleague.org)
- `status` — new, discussing, dates_proposed, confirmed, completed, cancelled
- `event_type` — private or public
- `external_registration_url` — optional URL to the host's own registration system (typically for public events at venues like libraries)
- `min_headcount` — minimum registrations required to run the event (private events)
- `assigned_instructor_id`
- `confirmed_date`
- `asana_task_id`
- `meetup_event_id` (if public)
- `google_calendar_event_id`
- `donation_link` — Give Lively URL
- `created_at`, `updated_at`

5.4 Registration / Interest

Created when an attendee clicks the shareable link and registers.

- `registration_id`
- `request_id` — links to the parent event request
- `attendee_name`, `attendee_email`
- `number_of_kids`
- `available_dates[]` — which of the proposed dates they can attend
- `status` — interested, confirmed, declined
- `created_at`

5.5 Email Thread

- `email_address` — the per-request address
- `request_id`
- `participants[]` — requester, admin(s), matched instructor(s), additional contacts (e.g. school principal)
- `messages[]` — stored inbound/outbound messages for AI extraction

---

6. Technical Architecture (Sketch)

This is a preliminary architecture. Detailed technical design is a separate document.

- **Web app:** Hosted on jointheleague.org subdomain or path (e.g. events.jointheleague.org). Server-rendered or SPA.
- **Database:** PostgreSQL for all app data (instructor profiles, event requests, registrations, email threads).
- **Auth:** Pike13 OAuth for instructors and admins only. Requesters and attendees do not have accounts — they access their requests and registration pages via obscure URLs sent in emails. Staff can see all requests when logged in and can share direct links. Request URLs are unguessable but not authenticated.
- **Email:** Amazon SES for inbound routing (per-request addresses via SNS/Lambda) and outbound (notifications, registration digests, iCal).
- **AI processing:** Email content sent to an LLM (Claude API or similar) for extraction. Results written to Postgres and Asana. No autonomous outbound actions.
- **Integrations:** Pike13 API (read availability, auth), Meetup API (create events, read RSVPs), Asana API (create/update tasks), Google Calendar API (create events, send invites), jointheleague.org content.json (read class data).
- **Content:** No local content storage. All class descriptions, images, and metadata pulled from content.json at render time or cached with short TTL.

---

7. Resolved Decisions

These were open questions that have been resolved:

Pike13 API write-back. Confirmed: the system will write bookings back to Pike13 when an instructor is assigned, so the event appears on their Pike13 calendar.

Meetup API access. Confirmed: we can create events and read RSVPs via the Meetup API.

Drive time calculation. Using zip code centroid distances with some accounting for time of day. Simple and sufficient for v1. May upgrade to a routing API later if accuracy becomes a problem.

AI extraction scope. Will use a lightweight model (Claude Haiku or similar). The AI's job is to recognize status transitions in email threads — e.g. "let's do it" triggers a move to the next pipeline stage — and extract action items. Conservative approach: only act on high-confidence extractions, flag ambiguous content for admin review.

Asana bidirectional sync. Yes. Status changes in Asana propagate back to the event app via Asana webhooks.

Date voting deadline. Configurable per event. The system will have a default (TBD) that admins can override when setting up the event.

Instructor consent. Required. When a request matches an instructor, they receive an email and must accept. The system sends reminders on a schedule. After a configurable timeout, the system moves to the next matched instructor.

Equipment tracking. The League's inventory system will support equipment reservation via API. This system will check equipment availability when showing dates and reserve equipment when an event is confirmed.

Multi-session events. Out of scope. This system handles single-session Tech Club events only. Multi-session courses involve instructor assignment for the full series and are paid programs — a different workflow.

Requester accounts. No accounts for requesters. Access is via obscure URLs sent in email. Staff (logged in via Pike13) can see all requests and share links. Request pages are open but unguessable.

Public event registration. No minimum headcount gate for public events. No date voting — the date is finalized during coordination before the Meetup event is created. Meetup RSVP is optional when a host has their own registration system.

External registration for public events. Hosts may optionally provide an external registration URL (e.g. a library's own event signup page). This link is included prominently at the top of the Meetup event description as a "Host Registration" hyperlink. The field can be provided at intake or added/edited later by an admin.

Registration digest emails. The system sends periodic registration summaries to the event email thread for public events, including a soft ask for hosts to share their own registration totals. This gives both sides visibility without requiring the host to use any League systems.

---

8. Remaining Open Questions

Instructor reminder cadence. How frequently should the system remind an unresponsive instructor before moving on? What's the timeout — 24 hours? 48?

Equipment reservation API. The inventory system integration is planned but the API contract isn't finalized yet. Needs coordination with inventory system development.

Meetup group selection logic. When a public event is created, the mapping from class topic to Meetup group needs to handle edge cases — classes that span multiple subgroups, or new classes that don't have a mapping yet.

Email thread participant management. Can participants be added to or removed from the email thread mid-conversation? What happens to thread continuity if someone is added late?

Donation link customization. Is the Give Lively link the same for all events, or can it be customized per event (e.g. pre-filled with the estimated cost)?

Registration digest cadence. How frequently should registration digests be sent? Weekly until the last week then daily? Fixed interval? Configurable per event?

RSVP sync frequency. The system will periodically sync Meetup RSVP data up until the day of the event. Exact polling interval TBD.

---

9. Suggested Phasing

Phase 1: Request Intake + Coordination

- Requestable class flag in content.json
- Request form on event app (including optional external registration URL)
- Instructor profiles in Postgres (topics, geography)
- Pike13 OAuth and availability reading
- Geographic/topic matching (zip centroid) and date display
- Instructor consent flow with reminders
- Per-request email address creation and routing (Amazon SES)
- Asana task creation
- Admin dashboard to view and manage requests

Phase 2: Registration + Event Finalization

- Private event registration with date voting
- Minimum headcount threshold logic
- iCal invite generation and delivery
- Google Calendar integration
- Public event: Meetup event creation via API (with external registration link support)
- Periodic Meetup RSVP sync
- Registration digest emails to event email thread
- Pike13 write-back: book instructor when event confirmed

Phase 3: AI + Automation

- AI email thread extraction (Haiku)

- Automated Asana updates from email content (including host registration counts)
- Asana bidirectional sync via webhooks
- Notification automation (reminders, deadlines, threshold alerts)
- Equipment reservation via inventory system API

Phase 4: Polish

- Instructor dashboard with event history
- Analytics and reporting
- Donation link customization
- Edge case handling (late participant additions, no-instructor-available workflows)